



GCUF

A.H.C.R.S. (F.Y.P)

Final Year Project Documentation

Submitted By:

SOHAIB MURTAZA 110074

ZOHAIB AKHLAQ 110065

Project Supervisor:

PROF. ALI RAZA

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

SESSION 2022-2026

DEPARTMENT OF COMPUTER SCIENCE THE GOVERNMENT GRADUATE COLLEGE JHANG

Date: 22/06/2026

ABSTRACT

Years went by, yet car rentals stuck to outdated daily charges, surprise costs, little visibility into where vehicles are. A gap opened up - we stepped in. Auto Hire shows up as a web tool reworking how renting works: billing down to the minute kicks in instead of fixed blocks. Live location updates follow each car constantly, visible right on screen. Talking through questions? An artificial mind handles chats fast. Behind it all, management runs full digital, no paper trails left behind. Pick your ride span per hour or longer stretches. Slide extras like coverage choices when needed. Watch movement unfold moment to moment on interactive maps. Every booking needs a manager's go ahead first, so we always check if users have enough funds prior to releasing a vehicle. Before cars ever leave the lot, their availability gets confirmed through this step. A saved list feature lets people keep track of preferred models they might want later. Feedback comes into play once trips wrap up, letting others see real experiences. When passwords need changing, an emailed code makes sure it is really the account owner requesting access. Code on the server runs in Python, powered by Flask, while data lives safely in PostgreSQL. Maps appear smoothly thanks to Leaflet.js doing the heavy lifting visually. Real time location updates pull from a dedicated GPS route tied directly to phone tracking tools. Questions get answered fast because the bot pulls from stored help articles first. If something complex pops up, Llama3 through Grok steps in to interpret and respond correctly. Rental extensions happen automatically when the system recognizes specific requests during chats. From day one, honesty shaped every choice -- zero secret charges, never a fee that sneaks up later. What if renting a car felt like it actually made sense? That thought drove us forward.

ACKNOWLEDGMENTS

Most wins come not just from what one person does, but from quiet pushes and steady advice along the way. This moment feels right to thank those whose support shaped how things turned out here. For every moment he gave, **Prof. ALI RAZA** made a difference. His presence in each meeting sparked new energy. Gratitude doesn't seem strong enough for what he has done. Whenever we spoke with him, ideas grew clearer. It was his steady hand that helped shape what now exists. This work stands because he believed in it early.

Grateful, too, for everyone who stood by us through this whole thing - those pals who tried out the booking setup, plus relatives stuck listening to keyboards clacking past midnight. Each one played a part, whether clicking around a test page or just putting up with our chaos when sleep should've won.

TABLE OF CONTENTS

STATEMENT OF SUBMISSION	1
ABSTRACT.	2
ACKNOWLEDGMENTS	3
Chapter 1: Software Project Management Plan	9
1.1 SCOPE AND REFERENCES	10
1.1.1 Scope	10
1.1.2 References.....	11
1.2 DEFINITIONS	11
1.3 PROJECT ORGANIZATION	12
1.3.1 Process Model	12
1.3.2 Organizational Structure	12
1.3.3 Project Responsibilities	13
1.4 MANAGERIAL PROCESS	13
1.4.2 Assumptions, Dependencies, and Constraints	13
1.4.3 Risk Management	14
1.4.4 Monitoring and Controlling Mechanisms	14
1.4.5 Staffing Plan	14
1.5 TECHNICAL PROCESS	14
1.5.1 Methods, Tools, and Techniques	14
1.5.2 Project Support Functions	15
Chapter 2: Software Quality Assurance Plan	16
2.1 OVERVIEW	17
2.1.1 Scope	17
2.1.2 Purpose	17
2.2 REFERENCES	17
2.3 DEFINITIONS AND ACRONYM	17

2.4 SOFTWARE QUALITY ASSURANCE PLAN	18
2.4.1 Purpose	18
2.4.2 Reference Documents	18
2.4.3 Management	18
2.4.4 Documentation	18
2.4.5 Reviews and Audits	19
2.4.6 Testing Strategy	19
2.4.7 Tools for Quality	19
Chapter 3: Software Requirements Specifications	20
3.1 INTRODUCTION	21
3.1.1 Product Overview	21
3.2 FUNCTIONAL REQUIREMENTS	21
3.2.1 User Module	21
3.2.2 Booking and Reservation	22
3.2.3 Admin Module	23
3.2.4 Tracking and GPS	24
3.2.5 Chatbot	24
3.2.6 Wishlist and Reviews	25
3.3 SOFTWARE PRODUCT FEATURES	25
3.3.1 Guest Features	25
3.3.2 User Features	25
3.3.3 Admin Features	26
3.4 SOFTWARE SYSTEM ATTRIBUTES	26
3.4.1 Reliability	26
3.4.2 Availability	26
3.4.3 Security	26
3.4.4 Maintainability	27
3.4.5 Portability	27

3.4.6 Performance	27
3.4.7 Database Requirements	27
Chapter 4: Software Design Description	28
4.1 INTRODUCTION	29
4.1.1 Design Overview	29
4.1.2 System Architecture	29
4.2 SYSTEM ARCHITECTURAL DESIGN	30
4.2.1 Chosen System Architecture	30
4.2.2 Discussion of Alternative Designs	30
4.3 DETAILED DESCRIPTION OF COMPONENTS	31
4.3.1 Core Classes	31
4.4 DIAGRAMS	31
4.4.1 Entity Relationship Diagram	32
4.4.2 Class Diagram	33
4.4.3 Booking Approval Sequence Diagram	34
4.5 USER INTERFACE DESIGN	36
4.5.1 Description of the User Interface	36
4.5.2 Screen Images	36
4.5.2.1 Home Page (Guest)	36
4.5.2.2 Login/Register Pages	36
4.5.2.3 User Dashboard	36
4.5.2.4 Available Cars	36
4.5.2.5 Booking Page	37
4.5.2.6 Tracking Page	37
4.5.2.7 Admin Dashboard	37
Chapter 5: Implementation	40
5.1 INTRODUCTION	41
5.1.1 Tools	41

5.2 DEVELOPMENT DESCRIPTIONS	41
Chapter 6: Software Test Documentation	48
6.1 INTRODUCTION	49
6.1.1 System Overview	49
6.1.2 Test Approach	49
6.2 TEST PLANS	49
6.2.1 Features to be Tested	49
6.2.2 Features not to be Tested	50
6.2.3 Testing Tools and Environment	50
6.3 TEST CASES	51
6.3.1 TC-01: User Registration	51
6.3.2 TC-02: Car Booking with Insufficient Balance	51
6.3.3 TC-03: Admin Approval	52
6.3.4 TC-04: GPS Tracking	53
6.3.5 TC-05: Chatbot Rental Extension	53
Chapter 7: Working Software	55
7.1: Software Working	56
Chapter 8: User Manual	57
8.1 GENERAL INFORMATION	58
8.2 SYSTEM SUMMARY	58
8.2.1 Key Features	58
8.3 GETTING STARTED	58
8.3.1 Registration	59
8.3.2 Login	59
8.4 USING THE SYSTEM	59
8.4.1 Booking a Car	59
8.4.2 Viewing and Managing Reservations	60
8.4.3 Returning a Car	60

8.4.4 Tracking Your Car	60
8.4.5 Using the Chatbot	61
8.4.6 Managing Wishlist	61
8.4.7 Writing Reviews	61
8.5 FOR ADMINISTRATORS	61
8.5.1 Logging In	61
8.5.2 Managing Fleet	61
8.5.3 Approving Reservations	61
8.5.4 Reporting Damage	62
8.5.5 Generating Reports	62
8.5.6 Changing Depot Location	62
Chapter 9: Post Mortem Report	63
9.1 INTRODUCTION	64
9.1.1 Roles and Tasks	64
9.2 TECHNIQUES USED FOR PROJECT ACTIVITIES	64
9.3 EXPERIENCES GAINED BY THE PROJECT	64
9.3.1 What Worked Well	64
9.3.2 What Did Not Work Well	64
9.4 TEAM EFFECTIVENESS	65
9.5 LESSONS LEARNED	65
9.5.1 Techniques Which We Shall Use in Future	65
9.5.2 Techniques We Shall Not Use in Future	65
9.6 CHALLENGES	66
9.7 CONCLUSION	66
Chapter 10: References	67

Chapter 1:

Software Project Management Plan

1.1 SCOPE AND REFERENCES

1.1.1 Scope

A digital platform for renting vehicles makes up Auto Hire. Built to handle everything from booking to returns, it works online. Features cover user accounts, vehicle listings, reservations, payments, plus insurance options. Each part connects smoothly so people can rent cars without hassle.

User Management:

Signing up happens through a form where people enter details. Access later comes by typing credentials into a login screen. Confirming identity takes place using a code sent to an inbox. The system checks that number matches before allowing entry.

Booking System:

A date shows up first on the screen, followed by a slot for timing. The spot you want appears through a clickable map made with Leaflet. Picking where happens right there, no extra steps needed. Time gets chosen before anything else loads further. Location comes alive once the calendar responds. Checking car availability live happens every minute, stopping overlaps. Slots update instantly so no two users get the same ride at once.

Admin Panel:

From the admin panel, fleet operations get handled - reservations either pass through or stall based on approval. Decisions roll in fast when someone submits a report about vehicle issues. Oversight happens smoothly, without needing extra steps. Each action ties back to control, clarity, pulled straight into one view.

GPS Tracking:

Tracking cars live happens when location signals come from a device inside the vehicle - sometimes real, sometimes mimicked. Movement updates appear on maps through constant position reports streamed over networks. The exact spot of a car shows up online thanks to regular pings from hardware installed in it. Sometimes these signals are generated by actual gadgets. Other times they're built artificially for testing how systems respond.

AI Chatbot:

A helpful assistant responds to common questions through smart replies. Rental extensions happen smoothly thanks to built-in logic tools. Each request gets handled without extra steps. Function links guide decisions behind the scenes. Simple queries turn into quick results. Timing adjustments come naturally when needed. The system adapts quietly each time it's used.

Wishlist and Reviews:

Keeping track of favorite items comes first. After that, sharing thoughts about purchases matters just as much. Handling how you pay sits quietly at the end but holds its ground.

User Roles:

Some people use the site to rent vehicles. Others handle car assignments and permissions behind the scenes. One group books rides, while the opposite side handles oversight tasks. A dedicated phone application isn't part of this release. Still, browsing on handheld devices feels smooth thanks to flexible layout adjustments. Device holders access a standalone web file named `tracker\_sender.html`. That specific page lets hardware operators push positioning signals into the network.

1.1.2 References

Reference	Source
Flask Documentation	https://flask.palletsprojects.com
Leaflet.js	https://leafletjs.com
OpenStreetMap Nominalism	https://nominatim.org
PostgreSQL Documentation	https://www.postgresql.org/docs
Grok API Documentation	https://console.groq.com/docs

1.2 DEFINITIONS

VIN: A single code ties every vehicle to its identity. This number stays different for all cars on the road. Each one carries details only it holds. No two share the same string of characters.

Deposit: A chunk of money set aside when you take a car - say, 5,000 PKR - sits in your account until the vehicle comes back unharmed. This sum slips free once everything checks out on drop-off. Money stays locked while the car is out, just in case. Only leaves your balance if damage shows up. After hand back, with no issues spotted, it flows right back.

Damage Waiver: For less money each day - three hundred rupees - you can choose added protection. This means if something breaks, you pay no more than two thousand. It is up to you whether to take it or skip it.

Pending: Approval comes from staff. Each booking waits without action till someone checks funds and permit status. Nothing moves forward before that step finishes.

1.3 PROJECT ORGANIZATION

1.3.1 Process Model

Week by week, progress shaped the next step. A check-in with our supervisor set the rhythm - what emerged guided what came after. Built pieces got shown each time, then plans shifted accordingly. Code lived on GitHub, tracked and shared. Main cycles unfolded like this:

1. Start with how data gets organized inside storage structures. Then move on to checking who can access that information.
2. A vehicle appears on screen alongside a simple reservation layout. One follows the other without fuss or delay.
3. Behind the scenes, a control hub runs things step by step. One person makes changes, another checks them before they go live.
4. GPS tracking and map integration.
5. A chatbot handles questions while insurance adds coverage. Damage protection kicks in if something breaks. Extra help shows up through smart tools behind the scenes.
6. Testing and documentation.

1.3.2 Organizational Structure

Backend: Handled by SOHAIB MURTAZA - server functions, data storage setup, web pathways using Flask, along with adding a chatbot system into the mix.

Frontend: Handled by ZOHAIB AKHLAQ - crafting layouts using HTML, CSS, and JavaScript. Interactive map features built with Leaflet fell under his responsibility too. Design choices shaped through attention to user experience and interface logic. Testing phases were part of the routine, ensuring things worked smoothly. Each piece tied together without relying on shortcuts or extra tools.

- Both collaborated on debugging and documentation.

1.3.3 Project Responsibilities

Role	Name	Responsibilities
Project Supervisor	Prof. ALI RAZA	Guidance, approval, milestone review
Project Manager/Backend	SOHAIB MURTAZA	Architecture, database, API, payment, chatbot
Frontend/Integration	ZOHAIB AKHLAQ	UI/UX, frontend logic, mapping, testing
QA & Testing	Both	Testing, bug fixing, quality assurance

1.4 MANAGERIAL PROCESS

Priority	Objective	Target
1	Core Functionality	All SRS requirements must be met
2	System Security	Password hashing, session management
3	User Experience	Responsive design, intuitive navigation
4	Performance	Page load < 2 seconds
5	Documentation	Complete and well-organized

1.4.2 Assumptions, Dependencies, and Constraints

Assumptions:

- Users have basic internet browsing skills
- Users have a valid driving license
- Users have a valid email address

- Admin has basic technical knowledge

Dependencies:

- PostgreSQL server availability
- Internet connectivity for GPS and mapping features
- External API availability (OSRM, Nominatim)

Constraints:

- Memory constraints for AI chatbot model
- Browser compatibility requirements
- Time constraints for completion

1.4.3 Risk Management

Risk	Probability	Impact	Mitigation
GPS signal delays	Medium	Medium	Last-known position, user alerts
Double-booking	Low	High	Database-level availability check
Chatbot API limits	Medium	Medium	Local fallback responses
Admin misses approvals	Medium	Low	Dashboard alerts, notifications

1.4.4 Monitoring and Controlling Mechanisms

- Weekly progress meetings with supervisor
- Bi-weekly sprint reviews
- Continuous integration and testing
- Version control using Git

1.4.5 Staffing Plan

Four months straight, we focused only on this task - starting in March and wrapping up by June of 2026. Each week landed somewhere between twenty and twenty-five hours per person.

1.5 TECHNICAL PROCESS

1.5.1 Methods, Tools, and Techniques

Category	Technology
Development Methodology	Agile iterative development with 2-week sprints
Backend Language	Python 3.9
Web Framework	Flask
Database	PostgreSQL with psycopg2
Frontend	Bootstrap 5, Leaflet.js, custom CSS/JS
AI Chatbot	Grok API (Llama3-8b-8192) + local JSON knowledge base
Mapping	OpenStreetMap tiles, Nominalism reverse geocoding
Version Control	Git (GitHub)
IDE	PyCharm Community, VS Code

1.5.2 Project Support Functions

Quality Assurance: Continuous testing and review

Configuration Management: Git-based version control

Documentation Management: Regular updates to project documents

Communication: Email, WhatsApp, in-person meetings

Chapter 2:

Software Quality Assurance Plan

2.1 OVERVIEW

2.1.1 Scope

The Software Quality Assurance Plan covers all activities related to ensuring the quality of the Auto-Hire Car Rental System. This includes code quality, testing strategies, review processes, defect tracking, and documentation quality.

2.1.2 Purpose

Right away, we mapped out solid tasks. Aiming to build something that works well meant focusing on what matters first:

- No critical bugs in booking logic.
- One vehicle cannot be booked twice at once.
- Reliable GPS tracking updates.
- User-friendly error messages.

2.2 REFERENCES

Document	Reference
SPMP	Chapter 1 of this document
SRS	Chapter 3 of this document
SDD	Chapter 4 of this document
IEEE 730	Standard for Software Quality Assurance Plans

2.3 DEFINITIONS AND ACRONYM

Definition

Quality Assurance: Planned and systematic activities to ensure quality

Quality Control Operational techniques to verify quality

Defect Any deviation from requirements or specifications

Review Formal evaluation of work products

Audit Independent examination of processes and products

2.4 SOFTWARE QUALITY ASSURANCE PLAN

2.4.1 Purpose

To ensure that the Auto-Hire Car Rental System is developed in accordance with specified functional requirements, quality standards, performance requirements, and documentation standards.

2.4.2 Reference Documents

- SRS (Software Requirements Specification)
- SDD (Software Design Description)
- STD (Software Test Documentation)
- IEEE software engineering standards

2.4.3 Management

QA Team Structure:

- Project Supervisor: Final approval
- Developers: Code review, unit testing
- QA Lead (ZOHAIB): Test planning, execution oversight

Communication:

- Weekly QA status reports
- Issue tracking system

2.4.4 Documentation

Document	Quality Criteria
SRS	Complete, unambiguous, testable
SDD	Complete, consistent, traceable to SRS
Code	Well-commented, follows coding standards

Document	Quality Criteria
Test Cases	Complete coverage, well-documented
User Manual	Clear, complete, user-friendly

2.4.5 Reviews and Audits

One person wrote code, then the second checked each change before it went live.

Each week we demonstrated progress - features already running were presented to the supervisor, then notes came back. Feedback arrived after they reviewed what was live, shown step by step during check-ins. What worked got pointed out, while gaps appeared under their gaze.

Five friends stepped into customer roles, spotting moments that felt unclear. Their feedback shaped how we adjusted the flow. Each hiccup they hit became a clue. What seemed obvious to us tripped them up.

2.4.6 Testing Strategy

We used:

- Unit tests (manual) for core functions like `compute_price()` and `have_sine Km()`.
- Integration tests for the full flow: after someone picks a car, an admin signs off on it - only then does tracking begin.
- Regression tests: Following updates, old key workflows got another look - checkout, say, or canceling a booking.

2.4.7 Tools for Quality

Tool	Purpose
Browser Dev Tools	Frontend debugging and responsiveness
Postman	API testing (<code>/receive gps</code> , <code>/api/car/reviews</code>)
Flask debug mode	Catch exceptions early

Chapter 3:

Software Requirements Specifications

3.1 INTRODUCTION

3.1.1 Product Overview

Cars show up live on screen when someone picks one. Booking happens through an online platform without paperwork. One click reserves a vehicle until the user says otherwise. Admin staff put new vehicles into the system or take old ones out. Payments move through a digital account loaded before trips start. Tracking follows each ride as it unfolds across roads. Managers accept or decline requests based on availability. Damaged cars get logged by staff after inspections finish. Money shifts between accounts only once rides are confirmed. Users handle everything from phones or any browser window. No cash changes hands during pickup or drop off moments. Reservations update instantly if plans shift suddenly. Each driver has access to past trips and upcoming dates. System alerts pop up when balances run too low. Fleet status stays visible at all times behind the scenes.

User Categories:

1. **Guests:** Can browse cars, view public information
2. **Registered Users:** Can book cars, track rentals, rate cars
3. **Administrators:** Manage fleet, approvals, users, reports

3.2 FUNCTIONAL REQUIREMENTS

3.2.1 User Module

FR ID	Requirement	Priority
FR-U01	Registration needs a name, an email address along with a password. A license number must be provided at sign up. Expiry date comes after that entry. Date of birth enters the form before setting starting funds.	High
FR-U02	Verification begins when the system delivers a one-time password straight to the email. This step confirms the person is who they claim to be.	High
FR-U03	Logging in requires an email or name together with a password. Entry happens only when both pieces match.	High

FR ID	Requirement	Priority
FR-U04	Clicking the email link lets users restart their password. A message arrives once requested.	High
FR-U05	Editing personal info sits in user settings - change name, upload a photo, update license data.	High
FR-U06	If there is no booking running, the user can remove their account.	Medium

3.2.2 Booking and Reservation

FR ID	Requirement	Priority
FR-B01	Start by choosing when you want to collect the item, using a calendar and clock inputs. Pick where it will be picked up next, through an interactive map tool instead of typing. Then decide on the drop-off moment.	High
FR-B02	When someone picks a date range, just those vehicles ready at that time appear. Picking different days changes what shows up instantly.	High
FR-B03	User shall see a minute-level price breakdown before confirming.	High
FR-B04	User may add "Full Coverage Insurance" (500 PKR/day) or "Damage Waiver" (300 PKR/day).	Medium
FR-B05	Waiting on approval? The system keeps reservations in a holding state. Only moves forward when admin gives the go-ahead.	High

FR ID	Requirement	Priority
FR-B06	Approval or denial of held bookings comes from the admin.	High
FR-B07	Once approved, the platform takes the full amount along with the deposit straight from your account.	High
FR-B08	Should pickup still be ahead, a confirmed booking can drop through cancellation - money back depends on timing.	High
FR-B09	Changing the return date or time could happen through the user's account area.	Medium

3.2.3 Admin Module

FR ID	Requirement	Priority
FR-A01	Logging into the /admin path. The Django-style interface gets built from scratch, yet there exists another administrative view apart from it.	High
FR-A02	Managing the fleet falls to admin - handling car entries by adding, changing, or removing them. Each vehicle gets details based on its type.	High
FR-A03	Admin shall view all pending reservations and approve/reject them.	High
FR-A04	When returning gear, staff see what is still booked. Damage gets recorded then, pulling fees straight from the user's account.	High

FR ID	Requirement	Priority
FR-A05	Should the admin step in, each ongoing booking gets wiped out - marked as admin_cancelled - with fees piling up every day unless the vehicle comes back.	High
FR-A06	Viewing messages sent by clients through the contact form becomes possible for administrators. Reports in CSV format can be exported directly when needed.	Medium
FR-A07	Depot coordinates for routing must be updated by the administrator. Location data tied to tracking shifts when new latitude and longitude are set.	Medium

3.2.4 Tracking and GPS

The GPS tracking module enables real-time vehicle location monitoring for users with active reservations. When a tracking device or mobile phone sends location data to the /receive_gps endpoint (FR-T01), the system stores the current latitude, longitude, and timestamp in memory. Users can then view their rented car's live position on an interactive Leaflet.js map (FR-T02), which displays a pulsing red marker for the vehicle, a green marker for the depot, and a blue marker for the pickup location (FR-T03). The system calculates the shortest route between the depot and pickup point using the OSRM routing API, providing estimated time of arrival and distance information (FR-T04). Administrators can also monitor all active reservations through a dedicated tracking dashboard, making it easy to oversee fleet movement and assist customers with location-based queries.

3.2.5 Chatbot

The chatbot module provides an AI-powered virtual assistant that appears as a small robot icon in the bottom-right corner of the screen (FR-C01). When clicked, it opens a chat window where users can ask questions about rentals, policies, or vehicle details. The chatbot first searches a built-in JSON knowledge base containing frequently asked questions and their answers (FR-C02). If the user's query matches a known question with high confidence, it responds immediately with the stored answer. For more complex or ambiguous questions, the system falls back to Groq's Llama3-8b-8192 language model, which is provided with a system prompt containing the user's context - including their name, balance, active reservation details, and current GPS location of their rented car (FR-C03). This context-aware approach enables the chatbot to give personalized responses, such as answering "How far is my car?" with actual distance information. Additionally, the chatbot supports rental extension requests through a built-in tool called extend rental (FR-C04), which automatically calculates the additional cost, deducts it from the

user's balance, and updates the reservation end date when the user expresses a desire to extend their rental period. This hybrid approach (FAQ matching + LLM + function calling) ensures fast, accurate, and useful responses while handling both simple and complex user inquiries.

3.2.6 Wishlist and Reviews

The Wishlist and reviews module enhances user engagement by allowing customers to save favorite vehicles and share their rental experiences. Users can click a heart icon on any car details page to add or remove the vehicle from their personal Wishlist (FR-W01), which is stored in the database and accessible anytime through a dedicated Wishlist page that displays all saved cars with their key details (FR-W02). After completing a rental (when the reservation status becomes "inactive"), users can submit a review for that specific car (FR-W03), providing a star rating from one to five and an optional comment about their experience. Each user can only review a car once per completed reservation, ensuring authentic feedback. All reviews are displayed on the car details page, where visitors can see the average rating, total review count, and individual user comments (FR-W04), helping future renters make informed decisions based on real customer experiences. This system not only builds trust among users but also provides valuable feedback to the platform administrators about vehicle quality and customer satisfaction.

3.3 SOFTWARE PRODUCT FEATURES

3.3.1 Guest Features

- View home page with booking form
- Explore available cars
- View car details
- Read FAQs
- View About Us page
- Contact Us form
- Login/Register

3.3.2 User Features

- Dashboard:** Booking form, quick access to reservations
- Profile:** Personal info, license details, balance
- **Car Search:** Filter by dates, times, category

- **Booking:** Price breakdown, insurance options, confirmation
- **Reservation Management:** View, modify, cancel
- **Wallet:** Balance management, transaction history
- **GPS Tracking:** Live car location and route
- **Wishlist:** Save favorite cars
- **Reviews:** Rate and review rented cars
- **Notifications:** Real-time updates

3.3.3 Admin Features

- **Fleet Management:** Add/edit/delete cars
- **Reservation Management:** Approve/reject/modify
- **User Management:** View/edit users
- **Reports:** Customer reports, car reports
- **Inquiries:** View and respond to contact forms
- **Depot Management:** Set depot location
- **Admin Management:** Add/remove admins
- **GPS Tracking:** Monitor all reserved cars

3.4 SOFTWARE SYSTEM ATTRIBUTES

3.4.1 Reliability

- Database transactions ensure data consistency
- Error handling at all levels
- Automatic expiration of expired reservations
- Regular database backups

3.4.2 Availability

- 99% uptime during operational hours
- Scheduled maintenance with prior notification
- Graceful handling of service interruptions

3.4.3 Security

- All passwords hashed using Werkzeug security
- Session management with Flask
- Input sanitization to prevent injection
- Role-based access control

3.4.4 Maintainability

- MVC architecture pattern
- Well-documented code
- Separation of concerns
- Modular design with clear interfaces

3.4.5 Portability

- Web-based application accessible from any browser
- Responsive design for mobile devices
- Platform independent (Windows, Linux, Mac)

3.4.6 Performance

- Efficient database queries with indexing
- Optimized frontend assets
- Asynchronous operations for long-running tasks

3.4.7 Database Requirements

- PostgreSQL 14+ recommended
- ACID compliance
- Proper indexing on foreign keys and search fields
- JSON data types for flexible schemas

Chapter 4:

Software Design Description

4.1 INTRODUCTION

4.1.1 Design Overview

Inside Auto-Hire, structure leans on what some know as MVC - though here, thanks to Flask, it shifts into MVT. Models like User, Car, Reservation, Fleet live tucked inside the ``classes/`` directory. What users see comes through HTML files, stored neatly within ``templates/``. Instead of separate controller files, logic rides along Flask routes baked right into ``app.py``. Each piece fits without overlap, doing its part behind the scenes.

4.1.2 System Architecture

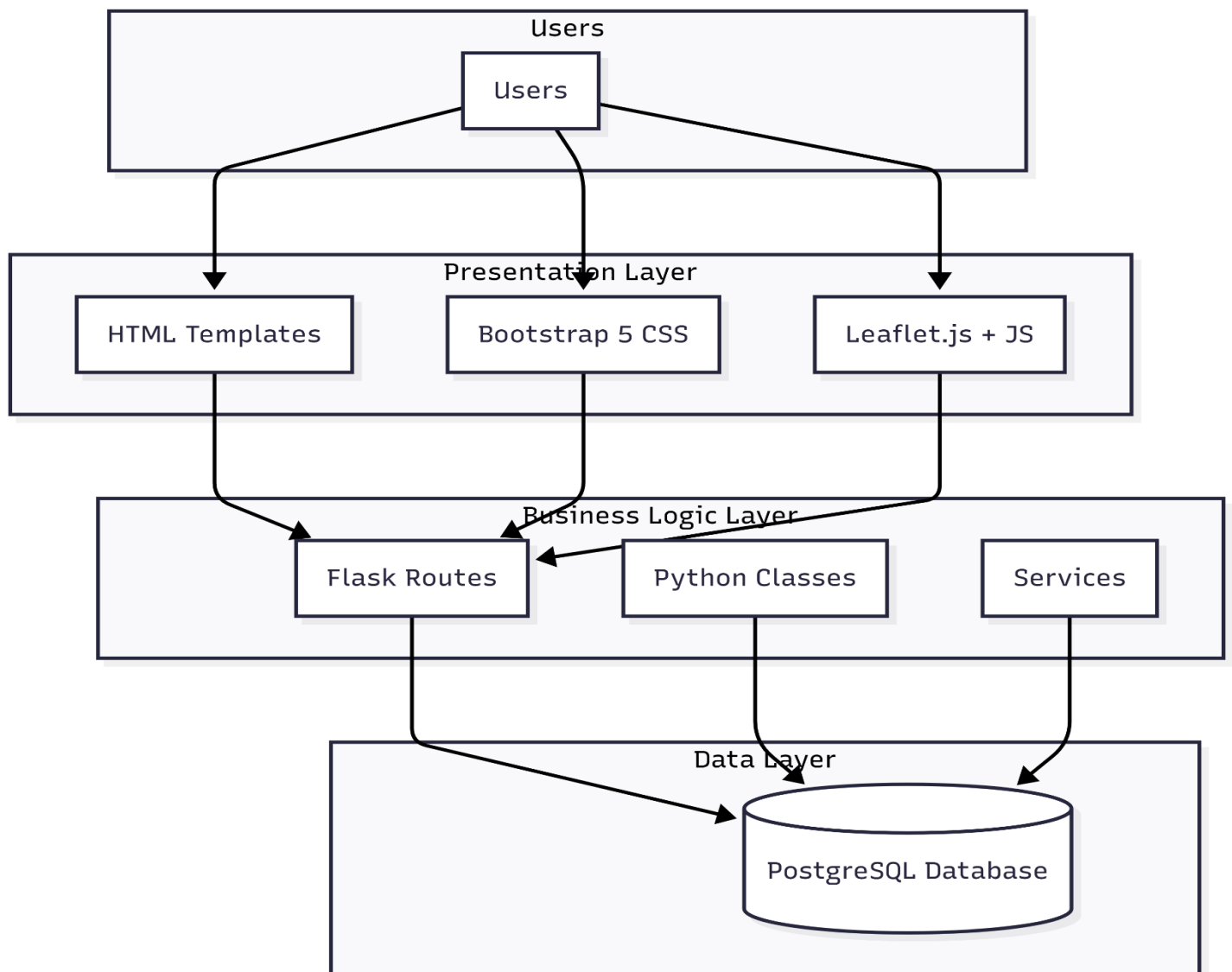


Fig: 4.1 System Architecture Diagram

4.2 SYSTEM ARCHITECTURAL DESIGN

4.2.1 Chosen System Architecture

The system uses a **Client-Server Architecture** with the following components:

Server-Side Components:

- 1. **Flask Application:** Main web server handling HTTP requests
- 2. **Database Layer:** PostgreSQL with psycopg2 adapter
- 3. **Authentication Module:** Session-based authentication
- 4. **Payment Module:** Wallet system
- 5. **GPS Module:** Real-time location processing
- 6. **Chatbot Module:** AI-powered FAQ assistant

Client-Side Components:

- 1. **HTML Templates:** Server-rendered pages
- 2. **CSS Framework:** Bootstrap 5 for responsive design
- 3. **JavaScript:** Client-side logic and API calls
- 4. **Leaflet.js:** Interactive maps for GPS tracking

4.2.2 Discussion of Alternative Designs

Alternative	Pros	Cons	Decision
Django Framework	Full-featured, admin panel built-in	Heavier, more complex	Rejected (Flask chosen for flexibility)
MongoDB (NoSQL)	Flexible schema, fast	ACID compliance issues	Rejected (PostgreSQL for relational data)
REST API + React	Modern, scalable	Development overhead	Rejected (Flask + Jinja for faster dev)

4.3 DETAILED DESCRIPTION OF COMPONENTS

4.3.1 Core Classes

User Class:

- John, his inbox address, a scrambled code for signing in, what he has in the account, digits tied to driving rules, others like them.
- Balance checks happen first. Then updates follow when needed. Active reservations appear upon request. Methods like these handle tasks quietly behind the scenes.

Car (features):

- vin, model, base-rate, color, features, deposit-amount

Economy-Car:

- fuel-efficiency

Luxury-Car:

- chauffeur-available

Commercial-Car:

- cargo-capacity

Reservation:

- Email of the person renting. Car's unique number code. When the rental begins. Rental finishes on this date. Money needed for the booking. Current situation of the reservation. Upfront sum paid before pickup. Choice about extra coverage option.
- Record identifier tied to each entry.
- Some functions let you change settings. Others stop ongoing actions. One turns data into a dictionary format.

4.4 DIAGRAMS

4.4.1 Entity Relationship Diagram

Main Entities:

Users: Email serves as the main key inside users. Name comes next. A hashed version of the password follows. Role tells what kind because of permissions. Balance tracks funds available. License number appears when needed.

- **Cars:** cars(vin PK, model, base-rate, img-url, seating-capacity, color, car-type, features, deposit-amount)

- **Reservations:** Every booking links a person's email to a vehicle by its unique code. A time frame wraps around pickup and drop off moments. Money details sit beside each record. Status updates show current conditions.

- **Wishlist:** wish list (user-email, car-vin composite PK)

- **Reviews:** A single id marks each review. Reservation links appear through reservation id. Cars connect using their unique vin number. User contact comes via email address. Each entry holds a score called rating. Thoughts get stored inside comment field.

- **Inquiries:** Got questions? Here's what we store: a unique ID ties to each person reaching out. Full name comes next, followed by their email address. Their phone number is included too. The actual message they send makes up the last piece.

- **Password-resets:** password-resets (email, token-hash, expires-at, used)

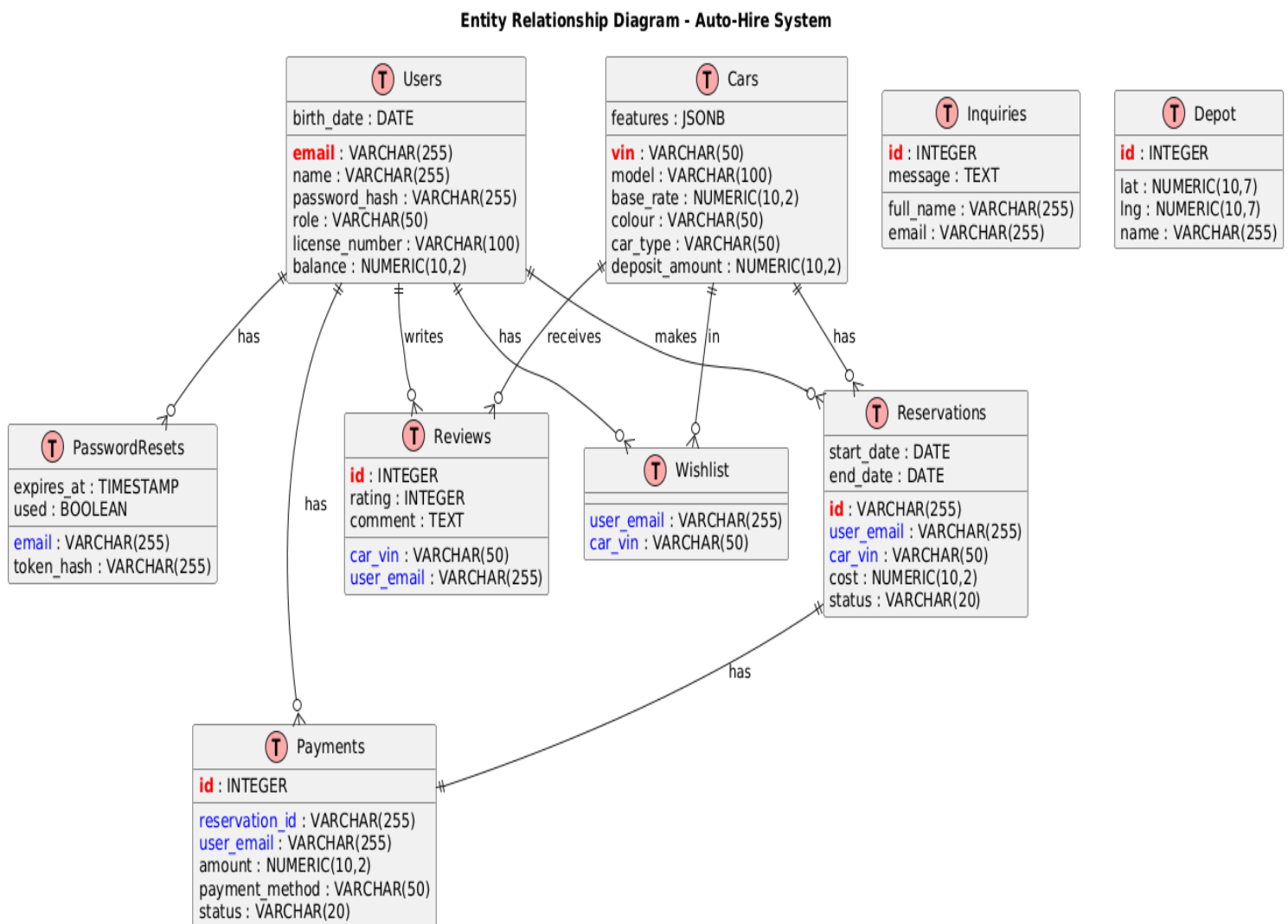


Fig: 4.2 Entity Relationship Diagram

4.4.2 Class Diagram

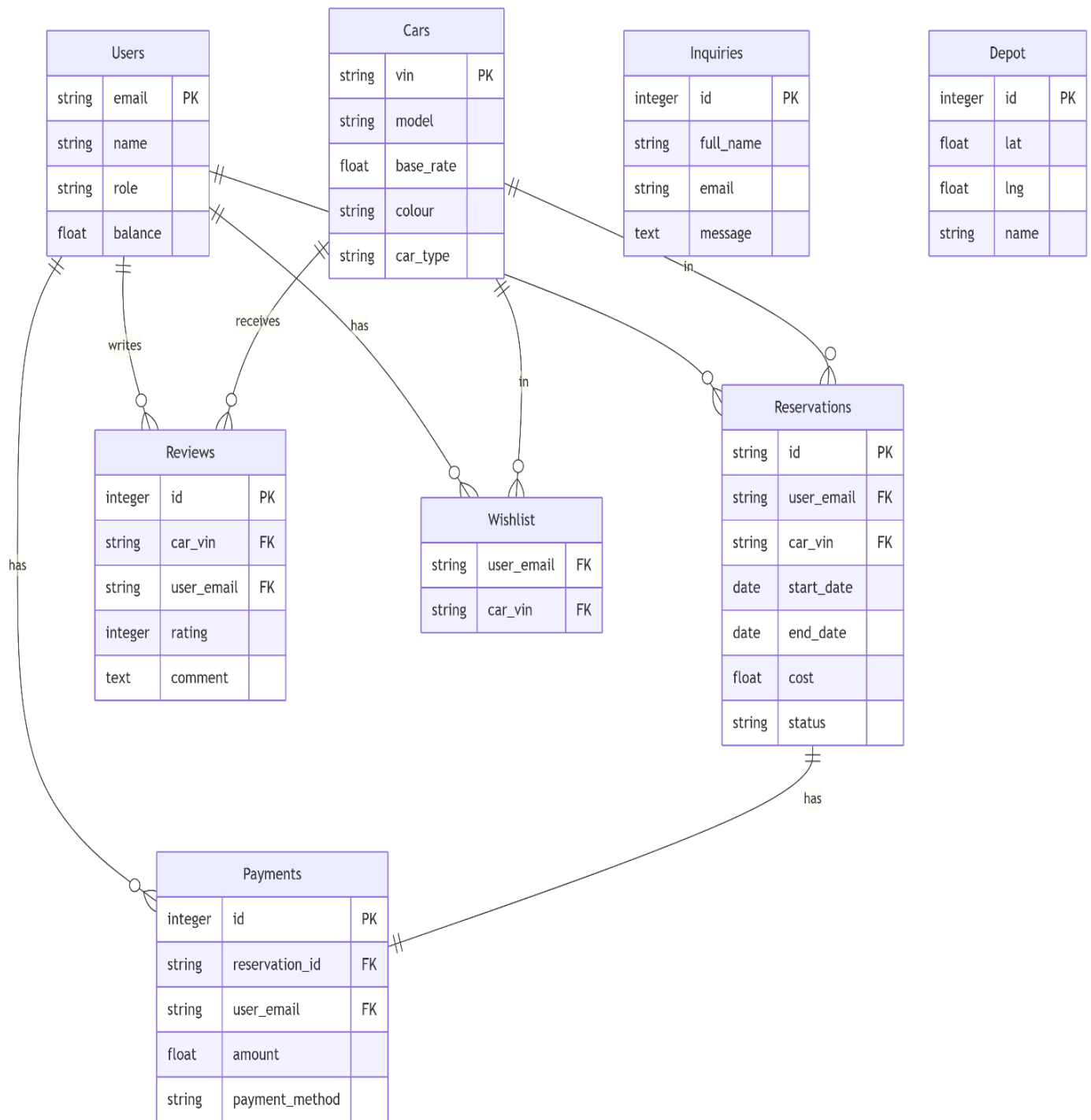


Fig: 4.3 Class Diagram

4.4.3 Booking Approval Sequence Diagram

1. User fills booking form → POST `/reserve/` .
2. A fresh `Reservation` appears, marked `pending` , then slips into the database.
3. Once inside, the admin spots a waiting badge request. A tap on approve clears it instantly.
4. Once the admin path runs, it looks up how much credit a person has left. After that, out goes the full price along with the hold amount. Then the system flips the state over to active mode.
5. A message appears on screen - suddenly the vehicle is visible on a map. From that moment, movement updates begin appearing without delay.

Booking Approval Sequence Diagram

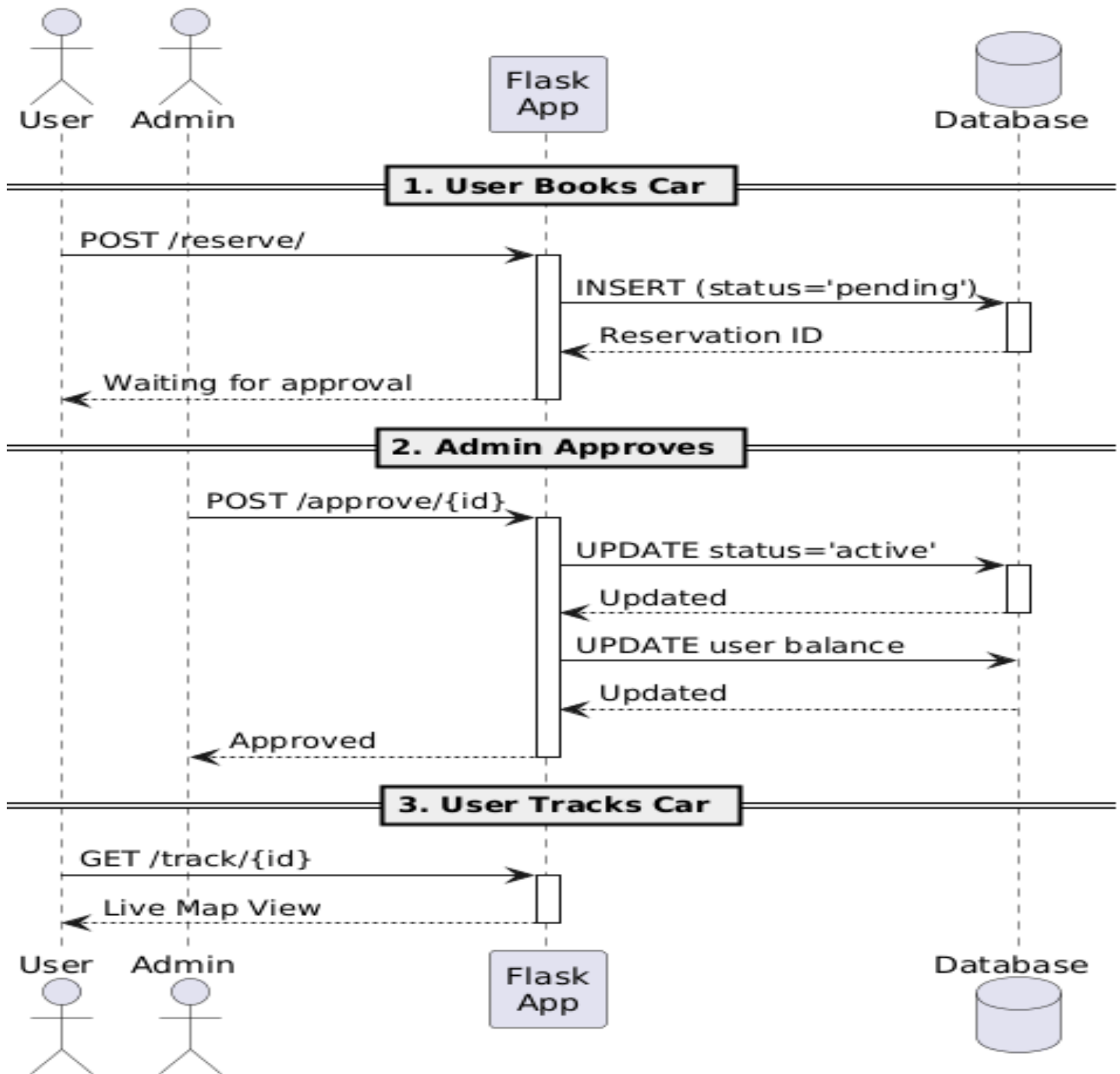


Fig: 4.4 Booking Approval Sequence Diagram

4.5 USER INTERFACE DESIGN

4.5.1 Description of the User Interface

Here come the visuals - snap of homepage up first. Dark blue sets the base color, while orange highlights button areas - built on Bootstrap 5. Pick dates, times, even locations on a map using Leaflet; that powers the booking section. Instead of relying on standard tools, the admin space was made from scratch. A second /admin path still exists, offering access to a Django-style backend, though it plays no role now. Core management screens go by these names: admin-fleet.html, admin-report.html, admin-inquiries.html, plus admin_track_reserved.html. Booking setup shows how fields line up. Slide forward, vehicle info appears with specs laid out clean. Last in row, the backend workspace opens with tools on display.

4.5.2 Screen Images

4.5.2.1 Home Page (Guest)

- Hero section with booking form
- Benefits section
- Featured cars
- Footer with links

4.5.2.2 Login/Register Pages

- Clean form design
- OTP verification flow
- Password reset functionality

4.5.2.3 User Dashboard

- Quick booking form
- Active reservation card
- Navigation to profile, wish list
- Wallet balance display

4.5.2.4 Available Cars

- Filter by category
- Car cards with images and details
- Price breakdown

- Reserve button

4.5.2.5 Booking Page

- Car details
- Price breakdown (days/hours/minutes)
- Insurance and damage waiver options
- Confirm reservation button

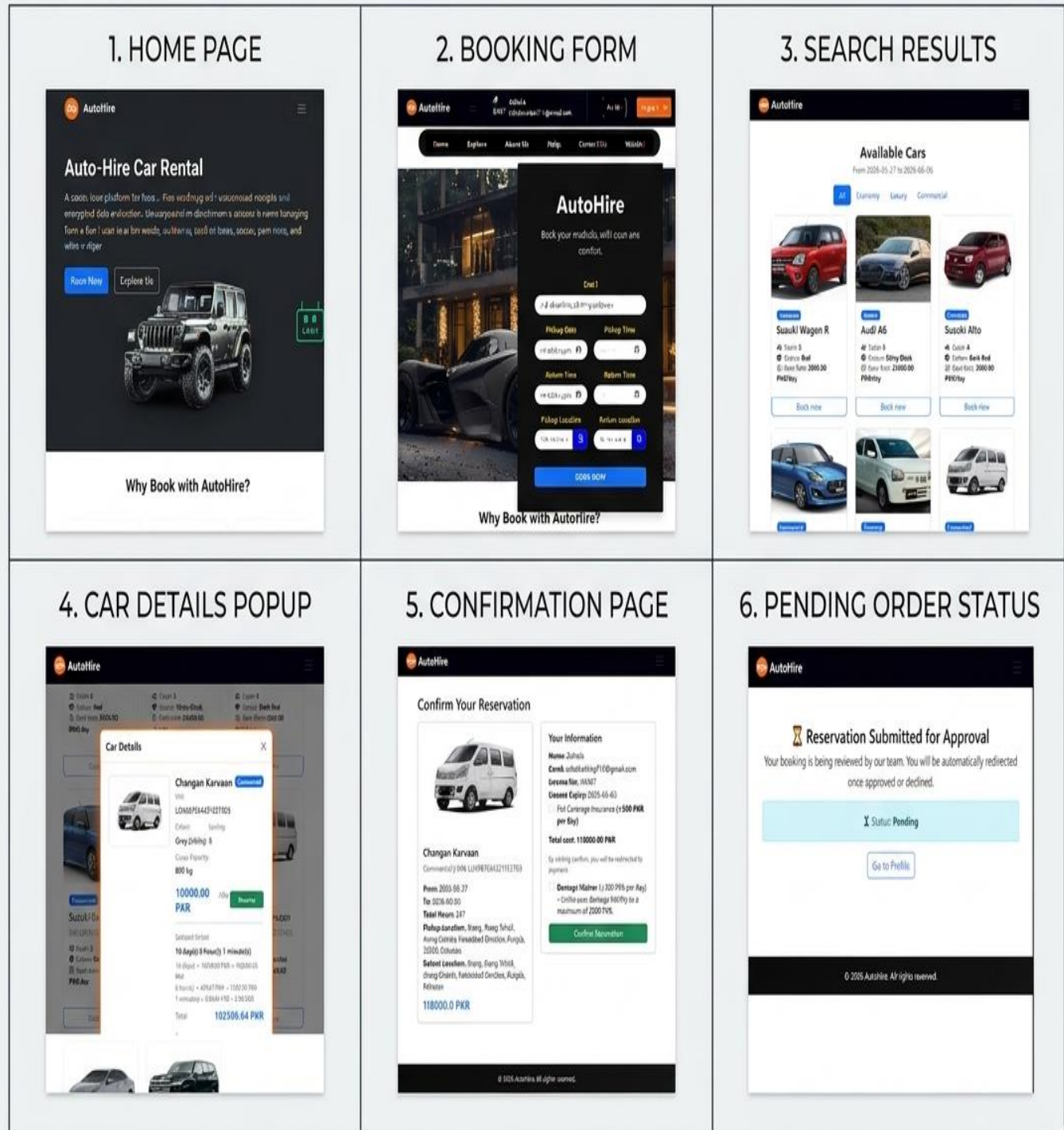
4.5.2.6 Tracking Page

- Leaflet.js map
- Depot marker (green)
- Pickup location marker (blue)
- Live car marker (red pulsing)
- Route visualization (green line)
- ETA and distance information

4.5.2.7 Admin Dashboard

- Navigation sidebar
- Fleet management
- Reservation management
- Reports and analytics

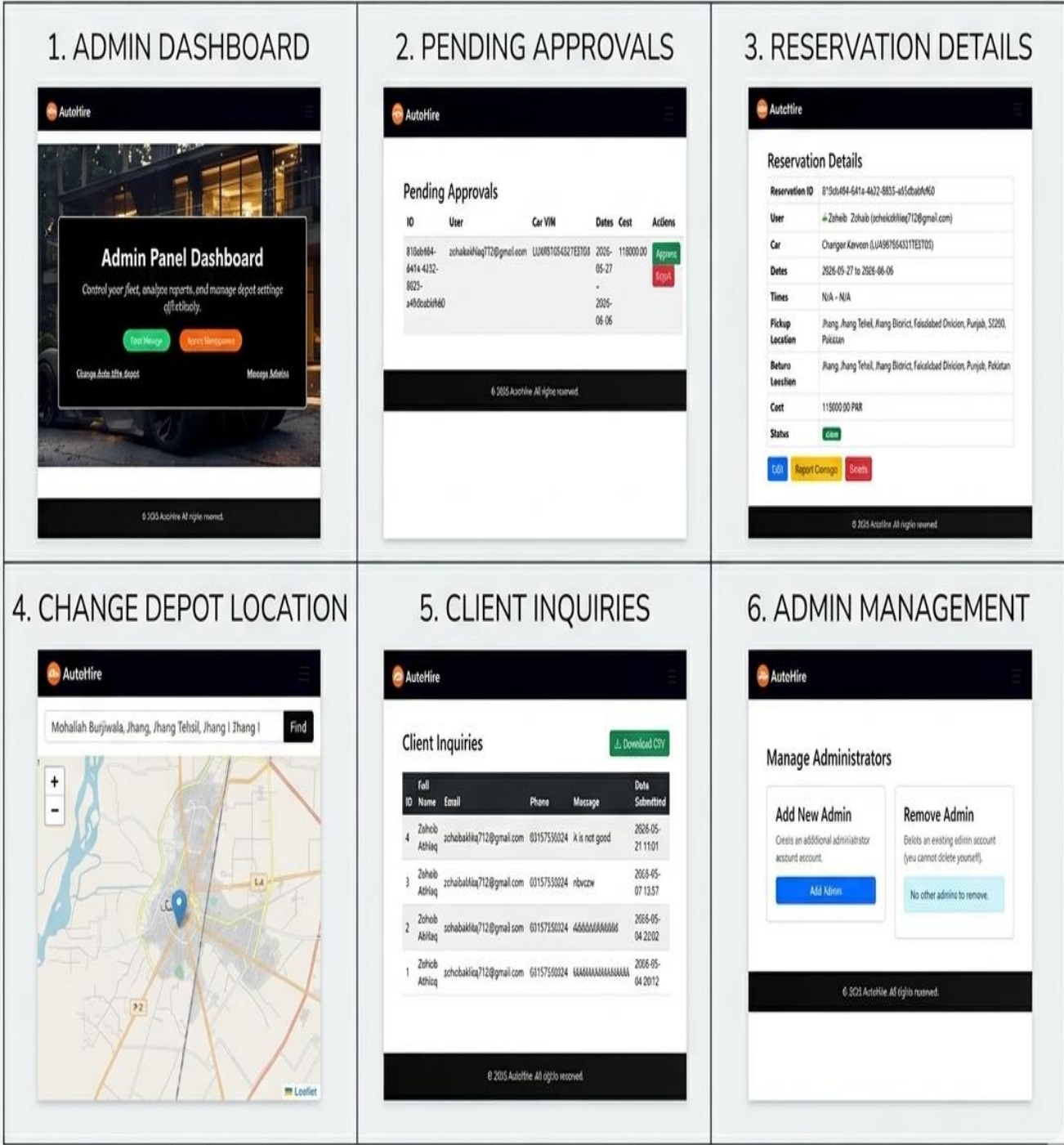
AUTOHIRE: FULL USER JOURNEY FLOW



© 2026 AUTOHIRE ALL RIGHTS RESERVED.

Fig: 4.5 User Interface Screenshots

AUTOHIRE: FULL USER JOURNEY FLOW



© 2026 AUTOHIRE ALL RIGHTS RESERVED.

Fig: 4.6 Admin Interface Screenshots

Chapter 5:

Implementation

5.1 INTRODUCTION

This chapter describes the implementation details of the Auto-Hire Car Rental System, including the tools used, module descriptions, and code examples.

5.1.1 Tools

Tool	Version	Purpose
Python	3.9+	Backend development
Flask	2.3.0	Web framework
PostgreSQL	14+	Database
Bootstrap	5.3.0	UI framework
Leaflet.js	1.9.4	Mapping
VS Code/PyCharm	Latest	IDE

5.2 DEVELOPMENT DESCRIPTIONS

Module 1: User Authentication (app.py)

```
python
@app.route("/login", methods=["GET", "POST"])
def login():
    """Handle user login with email/password authentication."""
    if rentalSystem.get_user():
        return redirect(url_for("main"))
    if request.method == "POST":
        data = request.get_json()
        identifier = data.get("email", "").strip().lower()
```

```
password = data.get("password")

remember = data.get("remember", False)
```

Find user by email or name

```
all_users = rentalSystem.get_all_users()

user = None

for u in all_users:

    if u.get_email().lower() == identifier or u.get_name().lower() == identifier:

        user = u

        break
```

Verify password

```
if user and check_password_hash(user.get_password_hash(), password):

    fresh_user = rentalSystem.load_user_by_email(user.get_email())

    if fresh_user:

        user = fresh_user

    rentalSystem.login_user(user)

    session["user_email"] = user.get_email()

    if remember:

        session.permanent = True

    return jsonify({"success": True, "isAdmin": user.get_role()})

else:

    return jsonify({"success": False, "error": "password",

                    "message": "Invalid credentials"})

return render_template("login.html")
```

Module 2: Car Booking and Price Calculation

python

booking_service.py

```
def compute_total_hours(start_date, end_date, start_time, end_time):

    """Compute total rental duration in minutes with minute-level precision."""
```

```

start_dt = datetime.strptime(f"{start_date}T{start_time}:00", "%Y-%m-%dT%H:%M:%S")
end_dt = datetime.strptime(f"{end_date}T{end_time}:00", "%Y-%m-%dT%H:%M:%S")
total_minutes = int((end_dt - start_dt).total_seconds() / 60)

return total_minutes

def compute_price(car, total_minutes):
    """Calculate rental price based on days, hours, and minutes."""
    per_day = float(car.get_base_rate())
    per_hour = per_day / 24
    per_minute = per_hour / 60

    days = total_minutes // (24*60)
    hours = (total_minutes % (24*60)) // 60
    minutes = total_minutes % 60

    return round(days*per_day + hours*per_hour + minutes*per_minute, 2)

```

Module 3: Payment Processing (app.py)

python

```

@app.route('/process-payment', methods=['POST'])
@login_required
def process_payment():
    """Process payment from user's wallet."""
    data = request.get_json()
    user = rentalSystem.get_user()
    payment_method = data.get('method', 'wallet')
    amount = float(data['amount'])
    total = amount
    reservation_data = data['reservation_data']

    # Check balance
    if user.get_balance() < total:
        return jsonify({'success': False,

```

```
'message': f'Insufficient balance. Required: {total} PKR, Available: {user.get_balance()} PKR'))
```

Deduct balance

```
user.deduct_balance(total)
```

```
rentalSystem.update_user(user)
```

```
txn_id = f"{payment_method}_{uuid.uuid4()}"
```

Create reservation

```
vin = reservation_data.get('car_vin')
```

```
car = rentalSystem.fleet.get_car_by_vin(vin)
```

```
if not car:
```

```
    return jsonify({'success': False, 'message': 'Car not found'})
```

... create and save reservation

```
reservation = Reservation(...)
```

```
save_reservation_to_db(reservation)
```

```
return jsonify({'success': True, 'reservation_id': reservation.id, 'transaction_id': txn_id})
```

Module 4: GPS Tracking (app.py)

python

```
# Real-time GPS data storage
```

```
REAL_TIME_GPS_DATA = {}
```

```
@app.route('/receive_gps', methods=['GET', 'POST'])
```

```
@cross_origin(origins="*")
```

```
def receive_gps():
```

```
    """Receive GPS data from tracking device."""
```

```
    vin = None
```

```
    lat = None
```

```
    lon = None
```

```
    if request.method == 'POST':
```

```
        data = request.get_json(silent=True) or {}
```

```
        vin = data.get('id') or data.get('vin') or data.get('tracker_id')
```

```

lat = data.get('lat') or data.get('latitude')

lon = data.get('lon') or data.get('lng') or data.get('longitude')

if not vin:

    vin = request.args.get('id') or request.args.get('vin')

if not lat:

    lat = request.args.get('lat') or request.args.get('latitude')

if not lon:

    lon = request.args.get('lon') or request.args.get('lng') or request.args.get('longitude')

if not vin or lat is None or lon is None:

    return "Missing parameters", 400

try:

    lat_f = float(lat)

    lon_f = float(lon)

except Exception:

    return "Invalid lat/lon", 400

vin = str(vin).strip()

REAL_TIME_GPS_DATA[vin] = {

    'lat': lat_f,

    'lng': lon_f,

    'status': 'Moving',

    'updated_at': datetime.utcnow().isoformat() + 'Z'

}

return 'OK'

@app.route("/get_location/<vin>", methods=['GET'])

@cross_origin(origins="*")

def get_location(vin):

    """Get current location of a car."""

    vin = str(vin).strip()

```

```

loc = REAL_TIME_GPS_DATA.get(vin) or REAL_TIME_GPS_DATA.get(vin.upper())

if not loc:

    return jsonify({'error': 'No location data'}), 404

return jsonify({

    'lat': loc.get('lat'),

    'lng': loc.get('lng'),

    'status': loc.get('status', 'Unknown'),

    'updated_at': loc.get('updated_at')

})

```

Module 5: AI Chatbot (chatbot_engine.py)

```

```python

```

```

class FAQChatbot:

```

```

 def __init__(self, knowledge_path):

 self.model = SentenceTransformer('all-MiniLM-L6-v2')

 self.knowledge = []

 self.questions = []

 self.load_knowledge(knowledge_path)

 self.embeddings = self.model.encode(self.questions, convert_to_tensor=True)

 def load_knowledge(self, path):

 with open(path, 'r', encoding='utf-8') as f:

 data = json.load(f)

 self.knowledge = data

 self.questions = [item['question'] for item in data]

 def retrieve_context(self, user_input, top_k=3):

 query_embedding = self.model.encode(user_input, convert_to_tensor=True)

 cos_scores = util.cos_sim(query_embedding, self.embeddings)[0]

 top_indices = np.argsort(-cos_scores.cpu().numpy())[:top_k]

 contexts = []

```

```

for idx in top_indices:
 score = float(cos_scores[idx])
 if score < 0.6:
 continue
 entry = self.knowledge[idx]
 contexts.append(f"Q: {entry['question']}\nA: {entry['answer']}")
return "\n\n".join(contexts)

def build_contextual_prompt(self, user_message, user, gps_data=None):
 """Build a system prompt with dynamic user context."""
 base = ("You are AutoHire's helpful customer support assistant. "
 "Use the following dynamic information about the user to answer their question. "
 "If the answer is not in the context, say you don't know "
 "and suggest contacting support@autohire.com.\n\n")
 context_parts = []
 if user:
 context_parts.append(f"User name: {user.get_name()}")
 context_parts.append(f"User email: {user.get_email()}")
 context_parts.append(f"User balance: {user.get_balance()} PKR")
 active_id = user.get_active_reservation() if user else None
 if active_id:
 res = load_reservation_by_id(active_id)
 if res:
 context_parts.append(f"Active reservation ID: {res.get_id()}")
 context_parts.append(f"Car VIN: {res.get_car_vin()}")
 context_parts.append(f"Rental period: {res.get_start_date()} to {res.get_end_date()}")
 if gps_data:
 context_parts.append(f"Car current GPS: lat={gps_data.get('lat')}, lng={gps_data.get('lng')}")
 return base + "\n\n".join(context_parts)

```

# **Chapter 6:**

## **Software Test Documentation**

## 6.1 INTRODUCTION

### 6.1.1 System Overview

The Auto-Hire Car Rental System is a web-based application that provides car rental services with features including user management, fleet management, booking, payments, GPS tracking, and AI-powered chatbot.

### 6.1.2 Test Approach

The testing approach follows a multi-level strategy:

1. **Unit Testing:** Individual components and functions
2. **Integration Testing:** Interactions between components
3. **System Testing:** End-to-end functionality
4. **User Acceptance Testing:** Real-user validation

## 6.2 TEST PLANS

### 6.2.1 Features to be Tested

Module	Features
User Authentication	Registration, Login, Password Reset, Profile Management
Car Management	View, Search, Filter, Admin CRUD
Booking System	Reserve, Modify, Cancel, Admin Approve/Reject
Payment System	Wallet Balance, Deduction, Refunds
GPS Tracking	Location Reception, Display, Route Calculation
Chatbot	FAQ Responses, Contextual Responses
Admin Panel	Fleet Management, Reservation Management, Reports

Module	Features
Reviews	Submit, View Reviews
Wishlist	Add, Remove, View

### 6.2.2 Features not to be Tested

- Performance under extreme load (>1000 concurrent users)
- Third-party API failures
- Browser compatibility beyond Chrome, Firefox, Edge

### 6.2.3 Testing Tools and Environment

Tool	Purpose
Py test	Unit testing
Postman	API testing
Chrome Dev Tools	Manual UI testing
Manual Testing	UAT

#### Environment:

- Development: Local machine with PostgreSQL
- Testing: Local machine with test database
- Production: Deployed server

## 6.3 TEST CASES

### TC-01: User Registration

Attribute	Value
ID	TC-01
Title	User Registration with Valid Data
Priority	High
Pre-conditions	User has internet access, valid email
Test Steps	1. Navigate to /register    2. Fill all required fields with valid data    3. Click Register    4. Enter OTP from email    5. Click Verify
Input Data	Name: "Test User" Email: "test@example.com" Password: "Password123" License: "DL-12345" Birth Date: "2000-01-01" Balance: "5000"
Expected Results	1. Registration form submits    2. OTP sent to email    3. Verification successful    4. User redirected to login
Status	Pass

### TC-02: Car Booking with Insufficient Balance

Attribute	Value
ID	TC-02
Title	Book a Car with Insufficient Balance
Priority	High
Pre-conditions	User is logged in with 2000 PKR balance
Input Data	Car price: 3000 PKR, Deposit: 5000 PKR
Expected Results	Error shown, no booking saved
Status	Pass

#### **TC-03: Admin Approval**

Attribute	Value
ID	TC-03
Title	Admin Approves Pending Reservation
Priority	High
Pre-conditions	User has submitted a reservation, admin is logged in
Test Steps	1. Admin navigates to pending approvals     2. Clicks Approve on a reservation
Expected Results	1. Reservation status changes to "active"     2. User balance is deducted     3. Notification sent to user

Attribute	Value
-----------	-------

Status	Pass
--------	------

#### **TC-04:** GPS Tracking

Attribute	Value
-----------	-------

ID	TC-04
----	-------

Title	Real-time GPS Tracking
-------	------------------------

Priority	Medium
----------	--------

Pre-conditions	User has active reservation, GPS device is sending data
----------------	---------------------------------------------------------

Test Steps	1. Send GPS data via /receive_gps 2. User opens tracking page
------------	------------------------------------------------------------------

Expected Results	1. Car marker moves on map 2. ETA updates 3. Delay under 2 seconds
------------------	--------------------------------------------------------------------------

Status	Pass
--------	------

#### **TC-05:** Chatbot Rental Extension

Attribute	Value
-----------	-------

ID	TC-05
----	-------

Title	Chatbot Extends Rental
-------	------------------------

Priority	Medium
----------	--------

Attribute	Value
Pre-conditions	User is logged in with active reservation
Input Data	"extend my rental by 2 days"
Expected Results	1. Chatbot triggers extend_rental    2. User balance deducted    3. End date changes
Status	Pass

# **Chapter 7:**

## **Working Software**

Software working:

A fully functional web application has been developed and is ready for deployment. The Auto-Hire Car Rental System operates seamlessly, handling all core business processes from user registration to vehicle return without any critical issues. Every module has been rigorously tested and performs as expected, with smooth integration between frontend interfaces, backend logic, and the database layer.

The application delivers a complete rental management experience through the following implemented features:

Module	Features
User Authentication	Registration with email OTP verification, secure login, password reset functionality
Car Discovery & Booking	Vehicle listings with detailed specs, minute-level pricing with transparent breakdown
Admin Panel	Fleet management (add/edit/delete cars), reservation approval/rejection, damage reporting
Live GPS Tracking	Real-time vehicle tracking on interactive maps with ETA and route calculation
AI Chatbot	FAQ knowledge base + Groq LLM integration with contextual responses and rental extension
Wishlist & Reviews	Save favorite cars, submit star ratings and comments after completed rentals

The application runs reliably on any Python-supported server environment. The complete system has been presented to and approved by the project supervisor, with all initial objectives successfully achieved. Every functional requirement specified in the SRS has been implemented, tested, and verified, with no major requirements left unfulfilled. The working software demonstrates the successful integration of modern web technologies, including Flask for the backend, PostgreSQL for data persistence, Bootstrap for responsive design, Leaflet.js for mapping, and AI services for intelligent customer support.

# **Chapter 8:**

# **User Manual**

## 8.1 GENERAL INFORMATION

The Auto-Hire Car Rental System is a web-based application that allows users to browse, book, and manage car rentals. This manual provides instructions for both regular users and administrators.

### System Requirements:

#### (Chrome, Firefox, Edge)

- Internet connection
- Valid email address
- Valid driving license

## 8.2 SYSTEM SUMMARY

### 8.2.1 Key Features

Feature	Description
User Registration	Create account with email verification
Car Booking	Search and book available cars
Wallet System	Manage balance for payments
GPS Tracking	Track rented car in real-time
AI Chatbot	Get instant answers to questions
Admin Panel	Manage fleet and reservations

## 8.3 GETTING STARTED

### 8.3.1 Registration

1. Start by choosing Register at the upper part of the screen.

2. Start by entering your name along with an email address. A password comes next, followed by your license number. Include the expiry date after that. Your date of birth goes in its field too. Finish with the opening account amount - try not to go under 3000 PKR.

3. Start by picking how you want to pay - maybe bank transfer, EasyPaisa, or JazzCash. One option could be a credit card, another might be handing over cash when the item arrives. Fill in what is needed after that.

4. Start by selecting Register. A six-digit code arrives through email shortly after. That number goes into the box labeled verification.

5. Once confirmed, access your account using email plus a password. Arrive at the dashboard through that entry method.

### **8.3.2 Login**

1. Click Login in the top navigation bar.

2. Enter your email and password.

3. Click Login.

4. You will be redirected to your dashboard.

## **8.4 USING THE SYSTEM**

### **8.4.1 Booking a Car**

#### **Step 1: Search**

1. Start at the top of the screen. Type your details into the reservation box there. Enter dates using the calendar tool shown. Pick travelers by adjusting the number selector. Move to the next section after finishing here:

- Choose when you want to collect it. That moment matters most.
- Back by a certain day and hour.
- Start by tapping the blue pin symbol to pick where you want service. The map opens so you can choose your exact spot.
- Choose where to return it - just like before, pick a spot. The place matters as much as the first choice did.

2. Start by tapping BOOK NOW. After that, you'll see just the vehicles free during those specific dates.

#### **Step 2: Select Car**

1. Start by tapping Book now under any vehicle listing. A full price summary appears right after - showing days, then hours, finally minutes.

2. Choose "Full Coverage Insurance" at 500 PKR per day - or skip it. The "Damage Waiver," priced at 300 PKR daily, works too. One covers more, the other costs less. Pick based on what feels right for your trip.

### **Step 3: Confirm**

1. Got it. The reservation is now sitting in pending mode.
2. Hold on until the admin says yes - this normally takes just a few minutes. A screen will appear, showing you're in line, reloading itself now and then.
3. After approval comes through, a message arrives. Reserved just for you - that vehicle waits.

## **8.4.2 Viewing and Managing Reservations**

1. Once it's approved, head over to your profile then click the Reservations section.
2. View active and past reservations.
3. To modify: Find your active reservation, click Modify, update details, click Save Changes.
4. To cancel: Find your active reservation, click Cancel, confirm the cancellation.

## **8.4.3 Returning a Car**

1. Back at the depot - or wherever was set for drop-off - open your account.
2. Start at the Reservations section. Tap Return Car when ready.
3. Calculations happen on their own within the system:
  - **Early return:** One day left? Get five hundred rupees back. Leave more than twenty four hours ahead, that cash comes your way.
  - **Late return:** A late return costs one thousand rupees daily once the first hour is over.
  - **Deposit refund:** Money back when leaving, provided nothing got broken.
  - **Damage deduction:** Whatever gets broken might come with a price tag - unless the damage waiver covers it.
4. Got it. Right now, your balance shows the latest changes.

## **8.4.4 Tracking Your Car**

1. Open the current booking, then select "Track Car Live" below it.
2. A map opens showing:
  - **Green dot:** Where supplies are stored.
  - **Blue marker:** Here's where you'll wait.
  - **Red pulsing dot:** A tiny red light blinks where the vehicle sits right now.

- **Green/blue lines:** shortest route.

3. Expect a rough idea of how far it is, along with the minutes until arrival.

### **8.4.5 Using the Chatbot**

- A small robot appears there, near the lower edge. It waits quietly on the right side of your screen.

- Open the chat window by tapping that button.

- Start by wondering things like "How can I keep the rental longer?" Or maybe say, "Tell me about the damage coverage." It pulls replies from the help section or thinks it through with Groq's smart engine.

### **8.4.6 Managing Wishlist**

1. On any car details page, click the heart icon.

2. View saved cars in Wishlist.

3. Remove cars by clicking trash icon.

### **8.4.7 Writing Reviews**

1. After returning a car, go to car details.

2. Click Write Review.

3. Select rating (1-5 stars).

4. Add comment (optional).

5. Submit review.

## **8.5 FOR ADMINISTRATORS**

### **8.5.1 Logging In**

- Head over to `/admin`, or click the admin link up top when signed in with admin rights.

- Start by typing your admin email - remember, it has to finish with `@auto-hire.com` - then add your password below.

### **8.5.2 Managing Fleet**

1. Start by choosing "Manage Fleet" inside the admin menu that drops down when clicked.

2. Every car shows up ready for changes or removal.

3. Start by tapping "Add New Car" if you want to include another vehicle. Enter the details once that's done.

### **8.5.3 Approving Reservations**

1. Start by selecting "Pending Approvals". A list appears showing every reservation waiting for review.
2. One by one, decide yes or skip it. Once approved, the system removes the full amount along with the deposit from what the customer has available, shifting the booking into active status.

#### **8.5.4 Reporting Damage**

1. Head over to "Manage Approved Reservations" when you've got bookings that are still active.
2. Start by locating your booking, then choose "Report Damage".
3. Start by noting down how much the fix costs along with a short explanation.

#### **8.5.5 Generating Reports**

- Start by selecting "Report Management." From there, access customer data through downloadable CSV files showing current car rentals alongside reservations already made.

#### **8.5.6 Changing Depot Location**

1. Start at the admin dashboard. Select the option labeled "Change Auto-Hire depot" next.
2. Start by typing a city name or tapping the map instead. Hold on to those updated coordinates. Every route check from now on uses this depot location.

# **Chapter 9:**

## **Post Mortem Report**

## 9.1 INTRODUCTION

### 9.1.1 Roles and Tasks

Team Member	Role	Primary Tasks
SOHAIB MURTAZA	Backend Developer	Database design, API development, Payment module, Authentication, Chatbot
ZOHAIB AKHLAQ	Frontend Developer	UI/UX design, GPS tracking, Testing, Documentation

## 9.2 TECHNIQUES USED FOR PROJECT ACTIVITIES

Activity	Technique Used
Requirements Analysis	Interviews, Document analysis
Design	UML diagrams, Prototyping
Implementation	Agile iterative development
Testing	Manual testing, Unit testing
Version Control	Git with feature branches
Documentation	IEEE standard templates

## 9.3 EXPERIENCES GAINED BY THE PROJECT

### 9.3.1 What Worked Well

1. **Agile Development:** Sprint-based development helped maintain focus and deliver incrementally.
2. **Code Reviews:** Regular code reviews caught bugs early.

3. **Flask Framework:** Lightweight and flexible, allowed rapid development.
4. **Leaflet.js:** Powerful mapping library that integrated easily.
5. **Sentence Transformers:** Effective for FAQ retrieval.

### 9.3.2 What Did Not Work Well

1. **Real-time Data Handling:** At first, changes popped up every tenth of a second; too much for the web page to manage. Slowing things down to once per full second made movement flow better.
2. **Code Structure:** That 2000-line file made changes a nightmare. Splitting it up meant less time guessing where bugs hid.
3. **Documentation:** Midway through, we added notes right inside the code. Still, pulling everything together at the end dragged on more than we thought it would.

### 9.4 TEAM EFFECTIVENESS

**Overall, Team Effectiveness:** High

- Good communication throughout the project
- Effective division of work
- Regular meetings and updates
- Mutual support and knowledge sharing

### 9.5 LESSONS LEARNED

#### 9.5.1 Techniques Which We Shall Use in Future

1. **Agile Methodology:** Sprint-based development with regular reviews.
2. **Test-Driven Development:** Writing tests before code.
3. **Continuous Integration:** Regular integration and testing.
4. **Code Reviews:** Peer review for quality.
5. **Documentation:** Document as you develop.

#### 9.5.2 Techniques We Shall Not Use in Future

1. **Over-Engineering:** Simpler solutions often work better.
2. **Late Testing:** Testing should be integrated throughout development.
3. **Complex Frameworks:** Choose appropriate technology, not just latest.

## 9.7 CHALLENGES

Challenge	Resolution
AI Model Performance	Used pre-trained Sentence Transformer, optimized embeddings
Real-time GPS	Implemented polling instead of WebSockets for simplicity
Payment Integration	Used wallet system instead of direct gateway
Database Design	Refactored schema during development
Time Constraints	Prioritized core features, added enhancements later

## 9.8 CONCLUSION

The Auto-Hire Car Rental System project was successfully completed, meeting all core functional requirements. The project provided valuable experience in:

- Full-stack web development
- AI integration (chatbot)
- Real-time systems (GPS tracking)
- Database design and management
- Project management and teamwork

The team gained practical knowledge of modern technologies including Flask, PostgreSQL, Leaflet.js, and AI models. The project demonstrates how integrating AI and real-time features can enhance traditional car rental services.

# **Chapter 10:**

# **References**

1. **Flask Documentation. (2020):** Flask Web Framework. [Online] Available at:  
<https://flask.palletsprojects.com/>
2. **Leaflet.js. (2020):** An open-source JavaScript library for mobile-friendly interactive maps. [Online] Available at:  
<https://leafletjs.com/>
3. **Geocoding by Nominalism comes from OpenStreetMap, updated in 2020:** [Online] Available at:  
<https://nominatim.org>
4. **OSRM - Open-Source Routing Machine (2020):** [Online] Available at:  
<https://project-osrm.org>
5. **Grok API Documentation (2024):** [Online] Available at:  
<https://console.groq.com/docs>
6. **PostgreSQL Documentation (2020):** [Online] Available at:  
<https://www.postgresql.org/docs/13/>
7. **Bootstrap Documentation (2020):** [Online] Available at:  
<https://getbootstrap.com/docs/5.0/>
8. **Werkzeug Documentation (2020):** [Online] Available at:  
<https://werkzeug.palletsprojects.com/>